

PC40 Lab 3 Report: Parallelizing a Convolutional Neural Network

Bora Yalçın

1 Overview

This is the report for Lab 3, where a sequential CNN for MNIST (convolution, ReLU, max-pooling, dense layer, softmax) is turned into a multithreaded C++17 program. Like Lab 2, this is not a machine-learning lab: the network and its math are given and untouched, my job is to find the independent work, split it between threads, keep the exact same numbers, and then measure and explain what I actually gained. I did this lab in C++17 again, for the same reason as Lab 2.

The lab sheet (PDF v21) asks for five short "syntheses" plus a final $\leq$ 2-page synthesis note instead of a full report, and says no full report is required. I am still writing this longer report, in the same format as Lab 1 and Lab 2, because I find it useful to keep; the actual deliverable the lab asks for is the separate `PC40_Lab3_Synthesis.tex` note. I kept every validated version as its own file, the same habit as last time. The optional extensions of this lab – filter/tile ownership, dynamic chunk scheduling, the lock-contention experiment, the input gradient dX, and loop ordering/tiling – were not attempted; I say so again at the matching part below.

2 Section 2 – Preparation and Smoke Test

2.1 Hardware, OS, toolchain

- **CPU:** Intel Core Ultra 9 285H, 16 cores, 1 thread per core (no SMT), hybrid design: 6 performance cores at 5.4 GHz, 8 efficiency cores at 4.5 GHz, 2 low-power efficiency cores at 2.5 GHz.
- **OS:** Kubuntu 26.04, kernel 7.0.0-38.
- **Compiler:** g++ 15.2.0 (Ubuntu 15.2.0-16ubuntu1), `-O2 -std=c++17 -Wall -Wextra -pedantic` for the sequential reference, `-pthread` added for every parallel file.
- **Numeric type:** `double` everywhere (the reference uses `double`, not `float` like Lab 2's network).
- **Datasets:** MNIST, linked from the Lab 2 data directory (`data/mnist`), and the supplied `-synthetic` generator for fast smoke tests.

This is the same laptop as Lab 2, so the hybrid core layout matters the same way here: 16 real cores, no hyper-threading, but not 16 equal cores. `lscpu -all -extended` confirms the three MHz groups (CPUs 0–5 at 5400 MHz, CPUs 6–13 at 4500 MHz, CPUs 14–15 at 2500 MHz), exactly as in the Lab 2 report. I come back to this in Part E and Part F.

2.2 My files

- `cnn_sequential.cpp`: the supplied reference, untouched.
- `cnn_parallel_v1.cpp`: Part B, image ownership, `forward_images_parallel()`.
- `cnn_parallel_v2.cpp`: Part B, row ownership, the C++17 `Barrier` class of Appendix A, batch-wide `BatchWS`, `conv_row`/`pool_row`/`dense_softmax`, `forward_rows_parallel()`.
- `cnn_parallel_train.cpp`: Part C, `batch_gradient_parallel()` and `train_epoch_parallel()`.
- `validate_D.cpp`: Part D, includes the reference and my three parallel files each in its own namespace and compares them, writing `validation.csv`.
- `cnn_parallel_final.cpp`: Parts E/F measurement binary, `-decomp image|rows -scope full|conv -pool -runs N`, timed training phases.
- `bench_E.py`, `plot_E.py`: the Part E benchmark driver (5 independent runs per configuration) and the plotting script, producing `bench_E.csv`, `KPI_E.md`, and the `E_*.png` figures.

Before writing the Part D validator I moved every parallel kernel into its own function (`forward_images_parallel`, `forward_rows_parallel`, `batch_gradient_parallel`) so the same code could be called from inside a namespace; I re-checked that the outputs did not change afterwards.

2.3 Worksheet

Checks. `./cnn_seq -self-test`:

Listing 1: ./cnn_seq –self-test

```
1 self-test 1 hand convolution      max_error=0.000e+00
2 self-test 2 forward vs naive      max_error=2.220e-16 (S 5.6e-17, P 0.0e+00, z 0.0e
      +00, sum(p)-1 2.2e-16)
3 self-test 3 pooling tie rule      wrong_winners=0
4 SELF-TEST PASS
5
```

and `./cnn_seq -gradcheck` passes with max error 2.259×10^{-10} (8 entries, $K[0]$, $K[7]$, $b[0]$, $b[1]$, $W[0]$, $W[169]$, $W[2000]$, $c[3]$, all against central finite differences with $\varepsilon = 10^{-6}$).

Model parameters. `struct Params`: K ($72 = 8$ filters $\times$ 9), b (8), W_d ($13\,520 = 10$ classes $\times$ 1352), c (10). Total 13 610 trainable parameters.

Per-image workspace. `struct WS`: S and A have 5408 entries each ($8 \times 26 \times 26$), P and `win` have 1352 each ($8 \times 13 \times 13$), z and p have 10 each.

Short training run. Synthetic, 2000 train / 500 test, 3 epochs, batch 32, lr 0.05, seed 42:

Listing 2: ./cnn_seq –synthetic –mode train –train 2000 –test 500 –epochs 3 –batch 32 –filters 8 –seed 42 –lr 0.05

```
1 epoch=1 train_loss=1.191873 test_loss=0.042122 accuracy=100.000000% time_s=0.100013
2 epoch=2 train_loss=0.019080 test_loss=0.009318 accuracy=100.000000% time_s=0.081170
3 epoch=3 train_loss=0.006749 test_loss=0.004887 accuracy=100.000000% time_s=0.077074
4
```

This README-documented value matches; 100% here is just the synthetic task being easy, not an MNIST result. The same command on real MNIST gives epoch-3 train_loss=0.480019, test accuracy 85.2%, about 0.08–0.09 s per epoch.

Why loss and time answer different questions. Loss tells me whether the model is computing/learning the right thing; time tells me how fast the program runs. My parallel version has to keep the exact same loss trajectory as the sequential one, because it is the same algorithm, and is only allowed to change the time. If the loss changed too, that would mean I had changed the computation, not just made it faster – which is exactly what Part D exists to rule out.

Note. My shell is zsh, so a quoted command-string variable has to be expanded with `${=ARGS}`, not `$ARGS` (the plain form failed on the first try with "unknown option").

2.4 Conclusion

Ready to analyse the computation: the sequential program builds, the two numerical checks pass, and I can locate the full forward and backward calculations and the per-image workspace in the source. This reference stays untouched for every later comparison.

3 Part A – Understanding the CNN and Identifying Parallelism

3.1 Objective

Before writing any thread I need to know the tensor layout, follow one forward pass by hand, and locate the dependency chain in the actual sequential source, so that I know exactly what each future worker will read, compute and write.

3.2 Answers

Identify the data. For $B = 32$ grayscale images the batch tensor is $[32, 1, 28, 28]$, 25 088 values. For 32 colour images of the same size it is $[32, 3, 28, 28]$, 75 264 values. Moving to the next image changes n (offset $+C{\cdot}28{\cdot}28$, i.e. +784 grayscale or +2352 colour); moving to the next channel changes d (+784); moving to the next pixel in a row changes j (+1, contiguous).

Compute by hand. With $X = \begin{bmatrix} 1 & 2 & 0 & 1 & 3 \\ 3 & 1 & 2 & 2 & 0 \\ 0 & 1 & 3 & 1 & 2 \\ 2 & 2 & 1 & 0 & 1 \\ 1 & 0 & 2 & 3 & 1 \end{bmatrix}$ and $K' = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$, zero bias, stride 1:

$$S' = \begin{bmatrix} 4 & -1 & 1 & 5 \\ 0 & 0 & 4 & -1 \\ 1 & 4 & 2 & 1 \\ 3 & -1 & -1 & 3 \end{bmatrix}, \qquad A' = \max(0, S') = \begin{bmatrix} 4 & 0 & 1 & 5 \\ 0 & 0 & 4 & 0 \\ 1 & 4 & 2 & 1 \\ 3 & 0 & 0 & 3 \end{bmatrix}, \qquad P' = \begin{bmatrix} 4 & 5 \\ 4 & 3 \end{bmatrix}.$$

Two checks: $S'_{0,0} = 0{\cdot}1 + 1{\cdot}2 + 1{\cdot}3 + (-1){\cdot}1 = 4$; $S'_{0,1} = 0{\cdot}2 + 1{\cdot}0 + 1{\cdot}1 + (-1){\cdot}2 = -1$.

Tie rule: the lower-right block of A in the main worked example is $\begin{bmatrix} 2 & 2 \\ 2 & 0 \end{bmatrix}$; three positions tie at value 2. "First maximum in row-major order" picks the top-left of that block, which is row 2, column 2 of the 4×4 array A, flat index $2 \cdot 4 + 2 = 10$. Sequential and parallel code must use the exact same rule even though P itself does not depend on it, because the winner index decides where the gradient flows in `backward_accumulate`: a different rule would move the gradient to a different input pixel, i.e. different training, not just a different-looking implementation.

Output sizes, $\lfloor (H + 2p - k)/s \rfloor + 1$ for a 28×28 input and a 3×3 filter: (a) stride 1, pad 0: 26×26; (b) stride 1, pad 1: 28×28; (c) stride 2, pad 0: $\lfloor 25/2 \rfloor + 1 = 13 \times 13$.

13 610 trainable parameters: convolution $8 \cdot 3 \cdot 3 = 72$ weights $+ 8$ biases $= 80$; dense $1352 \cdot 10 = 13\,520$ weights $+ 10$ biases $= 13\,530$ (with $1352 = 8 \cdot 13 \cdot 13$ the flattened pooled size); $80 + 13\,530 = 13\,610$.

Audit the sequential implementation. Deterministic small case, `-synthetic -mode train -train 32 -test 100 -epochs 1 -batch 32 -filters 8 -seed 42 -lr 0.05`:

Listing 3: One batch of 32, all images use the initial parameters θ_0

```
epoch=1 train_loss=2.299096 test_loss=2.294569 accuracy=28.000000% time_s=0.002012

```

Because one batch of 32 images is exactly one epoch here, 2.299096 is the initial loss at θ_0: close to $\ln 10 \approx 2.303$, which is what random small weights give for roughly a 1/10 chance per class.

Loops in `cnn_sequential.cpp`: convolution+ReLU l.76–89, pooling l.90–106, dense layer l.107–114, softmax l.115–121, `backward_accumulate` l.131–161 (dense gradient l.135–143, pooling backward l.144–145, convolution gradient l.146–160), `sgd_update` l.163–173, `train_epoch` l.243–258.

One-image offsets, $\mathrm{si}(f, r, q) = (26f + r)26 + q$, $\mathrm{pi}(f, r, q) = (13f + r)13 + q$:

Element	one-image offset	batch-wide, $n = 3$
$S[0, 0, 0]$	0	16 224
$S[2, 5, 7]$	1489	17 713
$S[7, 25, 25]$	5407	21 631
$P[0, 0, 0]$	0	4056
$P[2, 5, 7]$	410	4466
$P[7, 12, 12]$	1351	5407

Table 1: One-image vs. batch-wide offsets ($n = 3$ adds $3 \cdot 5408 = 16\,224$ for S, $3 \cdot 1352 = 4056$ for P)

Dependency chain for one image: $x \to S \to A \to P(+\,\mathrm{win}) \to z \to p$. The arrays that must survive until backward propagation are p (used in $dz = p - \mathrm{onehot}$), P (used for G_W), `win` (used by pooling backward), S (used as the ReLU mask), and x (used for G_K). A and z are not read again by backward – they are only a step on the way to P and p. All of these live in one per-image workspace, which is exactly why a parallel worker needs its own private copy of `WS`.

3.3 Results

The forward computation in `cnn_sequential.cpp`, unchanged (this is the loop I will later split across workers):

Listing 4: cnn_sequential.cpp, forward() l.74–90, the inner accumulator and the output of one convolution position

```
for (int r = 0; r < OH; ++r)
    for (int q = 0; q < OW; ++q) {
        double a = p.b[f];
        for (int u = 0; u < 3; ++u) {
            int xb = (r + u) * W + q, ko = kb + u * 3;
            a += p.K[ko] * x[xb] + p.K[ko + 1] * x[xb + 1] + p.K[ko + 2] * x[xb +
    2];
        }
        int ii = si(f, r, q);
        w.S[ii] = a;
        w.A[ii] = a > 0 ? a : 0;
    }

```

One output position, one accumulator `a`*, three multiply-adds per filter row*: this is the arithmetic grouping Part D requires every parallel decomposition to reproduce bit for bit, since changing the order of the 9 products changes the last bits of a `double`.

3.4 Conclusion

Checkpoint – Synthesis 1/5. Forward pipeline $X \to S \to A \to P \to h \to z \to p$: $28\times28\times1 \to 26\times26\times8 \to 26\times26\times8 \to 13\times13\times8 \to 1352 \to 10 \to 10$, trainable parameters K, b (conv) and W_d, c (dense), 13 610 total. Backward propagation needs a reduction wherever a parameter is reused: G_K, G_b sum over all 26×26 positions and over all images of the batch (Part C, Eq. 5–6); G_{W_d}, G_c sum over images. My proposed decomposition is image ownership (v1): worker t owns a contiguous range of images and the one-image workspace for them; shared read-only are the input images and θ; the only synchronization is the join after all workers finish; no lock is needed for the shared reads because nothing writes θ or the images during a forward pass.

Everything that matters for parallelizing the forward pass is already visible on paper: the same three nested loops (conv/ReLU, pooling, dense+softmax) run once per image, each output position has one well-defined formula, and nothing one image computes depends on another image. Backward propagation needs the saved S, P, `win`, p and x, all private to one image, which tells me a parallel worker only needs to own a per-image workspace, not touch anyone else's.

4 Part B – First Explicit Multithreaded Implementation

4.1 Objective

Build and validate two correct multithreaded forward decompositions from the same sequential reference: image ownership (v1) and row ownership (v2), deciding for each one what is shared, what is private, and where synchronization actually has to happen.

4.2 Answers

Prove the partition. Equation (2), $I_t = [\lfloor tM/T \rfloor, \lfloor (t+1)M/T \rfloor)$. For $M = 10, T = 3$: $[0,3)\,[3,6)\,[6,10)$, sizes $3,3,4$. For $M = 3, T = 5$: $[0,0)\,[0,1)\,[1,1)\,[1,2)\,[2,3)$, sizes $0,1,0,1,1$. The end of chunk t equals the start of chunk $t{+}1$ by the same formula, the first chunk starts at 0, and the last ends at $\lfloor TM/T \rfloor = M$, so the intervals tile $[0, M)$ with no gap and no overlap; sizes differ by at most one, and chunks are empty when $T > M$. I use `1LL*` products everywhere this is computed, to stay in 64-bit before the division.

v1 (image ownership) and v2 (row ownership) design. Worker t in v1 owns images I_t and runs the whole forward pass (conv → pool → dense → softmax) for them, in its own one-image `WS`; its only write to shared memory is its own images' class-0 probability slot. Worker t in v2 owns a chunk of a flat row index three times, once per phase: $M_1 = B{\cdot}8{\cdot}26$ convolution rows → barrier → $M_2 = B{\cdot}8{\cdot}13$ pooled rows → barrier → $M_3 = B$ images for dense+softmax. v2 therefore needs batch-wide `BatchWS` buffers for $S, A, P, \text{win}, z, p$ (about 51 MB at $B = 500$) instead of one small workspace per worker. Both v1 and v2 re-create their worker threads at every call/repetition; neither keeps a persistent pool at this stage (that only appears in Part E's `-pool` option).

Correctness so far. Checksum 500.023634460 for $T = 1, 2, 3, 4, 8, 16$ and $T = 501 > B$, in both v1 and v2; uneven case `-test 7 -threads 3` gives 2.108497536, same as sequential. For v2 I also compared every element of $S, A, P, \text{win}, z, p$ against the sequential forward (37 images, $T = 5$): zero differences. ThreadSanitizer reports no warnings for either file.

Checkpoint – correctness before timing. One input read by two workers with no lock needed: in v1, the convolution filter $K[0]$ (images themselves are never shared – each image has exactly one owning worker); in v2, also the input pixels, e.g. $X[n, 13, q]$ is read by convolution

rows $r = 11, 12, 13$, which can land on different workers (the halo of Fig. 5 of the lab PDF). No lock is needed because nobody *writes* X or K during a forward pass – a data race needs at least one write, and many threads reading the same unchanged value concurrently is simply safe. One output written by exactly one worker: v1's `prob0[i]` and its `ws[tid]`; v2's `S[n,f,r,q]`, owned by whichever worker holds row (n, f, r). The private accumulator is the local `double a` inside the convolution inner loop (or `conv_row`): it is born at `b[f]`, accumulates 9 products, is written once into `S`/`A`, and then its lifetime ends – it never crosses a thread boundary. What makes a completed layer visible to its consumer differs between the two designs: in v1 the same thread does conv, pool and dense for its own image, so there is nothing to "make visible" between layers, only the final `join()` publishes the results to the main thread; in v2 the producer of a row of S and the consumer of that row in pooling can be two different threads, so `bar.arrive_and_wait()` between phases is what makes the writes visible (the mutex inside the barrier gives the happens-before relationship), and then `join()` at the very end.

4.3 Results

Listing 5: cnn_parallel_v1.cpp, forward_images_parallel(): one join, one private workspace per worker

```
void forward_images_parallel(const vector<Sample> & te, const Params & p, vector<WS
    > & ws,
                             vector<double> & prob0, int T) {
    const int M = (int) te.size();
    vector<thread> workers;
    workers.reserve(T);
    for (int tid = 0; tid < T; ++tid)
        workers.emplace_back([&, tid] {
            const int lo = int((1LL * tid * M) / T);       // Equation (2)
            const int hi = int((1LL * (tid + 1) * M) / T);
            WS & w = ws[tid];                              // private workspace
            for (int i = lo; i < hi; ++i) {
                forward(te[i].x, p, w);                    // p, te: shared read-
    only
                prob0[i] = w.p[0];                         // exactly one writer
    per slot
            }
        });
    for (auto & th : workers)
        th.join();                                         // all images done and
    visible
}

```

Listing 6: cnn_parallel_v2.cpp, forward_rows_parallel(): 3 phases, 2 barriers, one join

```
void forward_rows_parallel(const vector<Sample> & te, const Params & p, BatchWS &
    bw, int T) {
    const long long M1 = 1LL * B * p.F * OH, M2 = 1LL * B * p.F * PH, M3 = B;
    Barrier bar(T);
    ...
    workers.emplace_back([&, tid] {
        chunk(M1, tid, lo, hi);                            // phase 1: conv rows
     -> S, A
        for (long long a = lo; a < hi; ++a) conv_row(te[n].x, p, bw, n, f, r);
        bar.arrive_and_wait();                             // all of S, A
    complete
```

```
9           chunk(M2, tid, lo, hi);                                  // phase 2: pooled
        rows -> P, win
10          for (long long a = lo; a < hi; ++a) pool_row(bw, p.F, n, f, r);
11          bar.arrive_and_wait();                                   // all of P, win
        complete
12          chunk(M3, tid, lo, hi);                                  // phase 3: images ->
         z, p
13          for (long long n = lo; n < hi; ++n) dense_softmax(p, bw, int(n));
14      });
15      for (auto & th : workers) th.join();                         // all probabilities
        complete
16  }
17
```

The reusable C++17 `Barrier` is the Appendix A class exactly, built around a mutex and a condition variable with a generation counter.

First timing observation (single runs, $B = 500$, 10 repeats, sequential 0.103 s): v1 at $T = 1$ is 0.108 s, $T = 4$ is 0.033 s (3.2×), $T = 16$ is 0.032 s, $T = 501$ is 0.110 s (back to roughly sequential: 5010 threads created for tiny work). v2 is slightly slower than v1 at every T here (e.g. $T = 4$: 0.044 s), because it pays 2 barriers and carries the big batch-wide buffers. Gains stop improving noticeably past $T \approx 4$ because one repeat is only around 30 ms and the create-per-call threads get re-created 10 times.

4.4 Conclusion

Checkpoint – Synthesis 2/5. v1 (image ownership): worker t owns a contiguous range of images and runs the full forward pass for them in its own `WS`, writing only its own images' probability slot; one synchronization point, the join. v2 (row ownership): worker t owns a chunk of a flat row index over the whole batch, writing into batch-wide buffers; two barriers (conv→pool, pool→dense) plus a join. Both match the sequential checksum exactly for $T = 1, 2, 3, 4, 8, 16, 501$ and an uneven case. First timing (B=500, 10 repeats, sequential 0.103 s): v1 reaches 0.032 s at $T = 16$ (about 3.2×); v2 is slightly slower at every T because of its extra barriers and buffers.

Both decompositions reproduce the sequential forward pass exactly, with the two designs costing different amounts of synchronization: one join for v1, two barriers plus a join for v2. v1 is simpler and cheaper when there is enough work per image; v2 exposes many more small tasks ($208B$ instead of B), which Part E shows actually helps when $B < T$. Filter and tile ownership, and dynamic (atomic-counter) chunk scheduling, were not attempted – they are the optional extensions of this part.

5 Part C – Parallel Training: Local Gradients and Reduction

5.1 Objective

Extend the validated image-owned forward pass (v1) into parallel mini-batch training that still computes exactly synchronous mini-batch SGD: all images of one batch use the same parameter snapshot θ_τ, gradients are accumulated privately, reduced once, and applied with exactly one update.

5.2 Answers

Design. Image ownership inside each batch: worker t gets images $[lo + \lfloor tB/T \rfloor, lo + \lfloor (t+1)B/T \rfloor)$, its own `WS`, and its own full-network gradient buffer G_t (13 610 doubles, K, b, W_d, c) plus a local loss accumulator. It zeroes G_t, then calls the unchanged `forward()`, `loss()` and `backward_accumulate()` (which *adds* into G_t) for each of its images. No worker ever calls

`sgd_update()`. After `join()`, the main thread reduces $G = G_0 + \cdots + G_{T-1}$ in increasing worker order (same order for the loss) and calls `sgd_update()` exactly once.

Lost update, simulated sequentially (Fig. 9 of the PDF). $g = 10$; worker 0 reads $r_0 \leftarrow g = 10$; worker 1 reads $r_1 \leftarrow g = 10$; $g \leftarrow r_0 + 2 = 12$; $g \leftarrow r_1 + 3 = 13$. Final value 13 instead of the intended $10 + 2 + 3 = 15$: both additions used the same stale value of g, which is exactly the read-modify-write race a shared accumulator would hit for real.

Would a mutex around the whole forward–backward–update sequence work? It would be free of data races, but it does two things wrong at once: (1) it changes the algorithm, because an update after every single image is per-image SGD – later images in the same "batch" would then see an already-updated θ, not θ_τ; and (2) it serializes all the useful work, since the lock covers the entire expensive computation, so only one worker ever runs at a time. Lab 2 Part F already showed that a full-call mutex can be slower than the sequential version; here it would additionally be the wrong algorithm, so I never built this variant.

Reductions inside convolution backward propagation. The same filter is reused at all 26×26 output positions, so $G_K[f,0,u,v] = \sum_n \sum_{r,q} \delta_{n,f,r,q} X_{n,0,r+u,q+v}$ and $G_b[f] = \sum_n \sum_{r,q} \delta_{n,f,r,q}$ are both a spatial sum over positions *and* a sum over images of the batch. Each worker performs its own spatial-plus-image sum locally while it owns that image, and the final reduction only combines the T worker totals. G_{W_d} and G_c need exactly the same ownership discipline; protecting only the convolution gradients would still leave the dense-layer gradients racing.

5.3 Results

Listing 7: cnn_parallel_train.cpp, the worker body, the fixed-order reduction and the single sgd_update()

```
1  double batch_gradient_parallel(const vector<Sample> & d, int lo, int hi, const
       Params & p,
2                                 vector<WS> & ws, vector<Grad> & G, vector<double> &
       local_loss,
3                                 Grad & g, int T) {
4      for (int tid = 0; tid < T; ++tid)
5          workers.emplace_back([&, tid] {
6              const int a = lo + int((1LL * tid * B) / T);           // Equation (2)
7              const int b = lo + int((1LL * (tid + 1) * B) / T);
8              G[tid].zero();                                         // gradients
       zeroed per batch
9              local_loss[tid] = 0;
10             for (int i = a; i < b; ++i) {
11                 forward(d[i].x, p, ws[tid]);                       // reads
       theta_tau only
12                 local_loss[tid] += loss(ws[tid], d[i].y);
13                 backward_accumulate(d[i].x, d[i].y, p, ws[tid], G[tid]);   // ADDS
       into G[tid]
14             }
15             // no worker calls sgd_update()
16         });
17     for (auto & th : workers) th.join();                           // all gradients
        at theta_tau done
18     g.zero();                                                      // fixed-order
       serial reduction
19     for (int t = 0; t < T; ++t) {
20         for (size_t i = 0; i < g.K.size(); ++i)  g.K[i]  += G[t].K[i];
21         ... // same for b, Wd, c
22         L += local_loss[t];
23     }
```

```
    return L;
}
// train_epoch_parallel(): tot += batch_gradient_parallel(...); sgd_update(p, g, lr,
    hi - lo);

```

5.4 Conclusion

Checkpoint – Synthesis 3/5. All images of batch τ must use the same θ_τ because the algorithm being preserved is synchronous mini-batch SGD: if θ changed mid-batch, different images would effectively be trained by different models and the gradient sum would not correspond to any real parameter snapshot. Gradients are accumulated locally in a private G_t per worker (each worker's own `backward_accumulate` calls only add into its own buffer). They are reduced by the main thread, serially, in increasing worker order, after `join()`. The single update happens once, right after that reduction, inside `sgd_update`.

Training is parallelized the same way as forward (image ownership), with the only new ingredient being a private full-network gradient buffer per worker plus one fixed-order reduction. All workers reading the same θ_τ causes no race because nothing writes θ during the batch; all workers writing their own G_t causes no race because the slots are disjoint; the reduction and the single update happen only after every worker has joined, so the whole thing stays synchronous mini-batch SGD by construction, not by locking. The contention experiment (shared buffer under one mutex, or one lock/atomic per parameter block) was not attempted – it is listed as an optional extension of this part.

6 Part D – Correctness Validation

6.1 Objective

Show, numerically, that changing the decomposition or the worker count did not change what is computed: exact equality for every forward value, and a tight tolerance for every gradient and parameter after a reduction.

6.2 Answers

`validate_D.cpp` compiles the untouched reference and my three parallel files, each in its own C++ namespace, and runs them side by side on deterministic synthetic and MNIST inputs. Comparison rule, exactly Equation (8) of the PDF:

Listing 8: validate_D.cpp, the two comparison rules used throughout

```
static const double EPS_ABS = 1e-10, EPS_REL = 1e-8;   // starting values from the
    lab (Eq. 8)
void exact(a, b, idx)  { /* a == b required; NaN fails */ }
void tol(a, b, idx) {
    double d = fabs(a - b), m = max(fabs(a), fabs(b));
    if (d > EPS_ABS + EPS_REL * m) fail(idx);           // Equation (8)
}

```

Forward values (v1, v2): exact. $S, A, P, \text{win}, z, p$ compared bit for bit (max error 0) for 1 image ($T = 1, 2, 4$), $B = 5, T = 3$ (uneven partition 1+2+2), $B = 3, T = 8$ ($T > B$, some workers empty), a controlled pooling tie, $B = 37, T = 5$, and an MNIST case $B = 5, T = 3$. Unwritten outputs would have been caught by sentinel initialization.

Gradients at the same θ_τ, before division (tolerance rule): bit-identical at $T = 1$; across every uneven partition tested, including $B = 32, T = 7$ (chunks 4+5+4+5+4+5+5)

and an MNIST case $B = 32, T = 5$, the maximum absolute error across all gradient blocks (G_K, G_b, G_{W_d}, G_c) is 3.775×10^{-15} and the maximum relative error is 2.130×10^{-12} – both many orders below the $10^{-10}/10^{-8}$ tolerance. No non-finite values anywhere. These are not bit-identical for $T > 1$ because the reduction groups the per-worker sums in a different order and floating-point addition is not associative.

One update: maximum parameter difference 1.388×10^{-17}. **3 epochs (2000 images, last batch 16):** same update count (189), epoch losses within 1.110×10^{-15}, parameters within 5.551×10^{-16}, for $T = 1, 3, 16$.

Checkpoint – one complete training step. The table below is one such check at $B = 32$, $T = 7$ (4+5+4+5+4+5+5 images per worker). Gradients are zeroed per batch inside each worker (`G[tid].zero()`) and again in the main reduction target (`g.zero()`); the division by the actual batch size happens exactly once, inside `sgd_update`, as `a = lr / B`; θ is only ever written after `join()`, so every image's gradient was necessarily computed from the same θ_τ. Good accuracy alone cannot catch a missing worker's gradient (it would just mean a slightly different, still plausible-looking, trajectory) or an update applied once per worker instead of once per batch (training would still proceed, just not as mini-batch SGD); only comparing the gradients and the post-update parameters directly, as above, can tell the two apart from the intended algorithm.

6.3 Results

Quantity	Case	Max error
$S, A, P, \text{win}, z, p$	all forward cases above	0 (exact)
G_K, G_b, G_{W_d}, G_c (abs)	worst over all 104 comparisons	3.775×10^{-15}
G_K, G_b, G_{W_d}, G_c (rel)	worst over all 104 comparisons	2.130×10^{-12}
One update, params	worst over all cases	1.388×10^{-17}
3-epoch run, params	$T = 1, 3, 16$	5.551×10^{-16}

Table 2: Part D correctness summary. `validation.csv` has 104 comparisons in total, 0 failures.

6.4 Conclusion

Checkpoint – Synthesis 4/5. Maximum numerical error observed: forward values exact (0) in every case; gradients 3.775×10^{-15} absolute / 2.130×10^{-12} relative, worst case over 104 comparisons including the non-divisible partition $B = 32, T = 7$ (4+5+4+5+4+5+5); one complete parameter update, max difference 1.388×10^{-17}.

The forward decompositions are not approximately correct, they are bit-for-bit the reference; the only place floating-point reordering is visible at all is in the gradient reduction, and there the worst error (gradients 3.8×10^{-15} absolute, parameters 1.4×10^{-17} after one update) sits nowhere near the 10^{-10} tolerance. That gap is what lets me trust the next two parts' timings: whatever I measure there is measuring the same algorithm as the sequential baseline.

7 Part E – Granularity and Scalability

7.1 Objective

Measure execution time against the number of workers T, for both decompositions, including the small-batch case $B < T$, and connect the observed curves to thread management, synchronization, reduction cost and memory.

7.2 Answers

Protocol. 5 independent invocations per configuration; one untimed warm-up repeat; the same timing scope as the sequential reference (ordered checksum accumulation stays inside the timer, per Section 2.4 of the PDF); when a persistent pool is used (`-pool`), its setup cost is reported separately; training parameters are re-initialized outside the timer between repeated measurements. Machine: the Ultra 9 285H above, 6P+8E+2LP-E, no SMT, so $T = 16$ uses 16 distinct physical cores and is not oversubscription by thread count – but it is still not 16 equal cores.

Inference, $B = 500$ (synthetic, 100 repeats; sequential 0.9153 s). Best result: v1, persistent pool, $T = 16$: 0.3205 s, $S_T = 2.84$ (2.86× vs. sequential), 156 023 img/s. Convolution alone (v1, create mode) is 0.6947 s at $T = 1$, 61% of the forward, and plateaus around 3× (0.2261–0.2625 s across T). Extra memory: v1 stays under 2 MB even at $T = 16$ (0.11–1.65 MB, one small `WS` per worker); v2 pays a flat 51.46 MB for its batch-wide buffers regardless of T. v1 beats v2 at this batch size, at every T, because there are enough images for every worker and v2 additionally pays 2 barriers plus the big buffers.

Inference, $B = 1 < T$ (30 000 repeats; sequential 0.5364 s, about 18 µs per image). No configuration speeds up. v1 cannot use extra workers at all once $T > 1$ (only one image exists, so one worker does all the work and the rest wait); its create-mode curve only gets slower with T (2.94 s at T=1 to 11.60 s at T=16), and even the pool, which removes thread creation, still slows down past $T = 1$ because every extra worker still has to cross two barriers for a 0.1 µs task. v2 *can* in principle spread a single image's 208 rows (104 conv + 104 pool rows for B=1, F=8) over more workers, but it pays the same 2-barriers-per-repeat tax on top, and ends up slower than v1 at every T in both create (19.44 s at T=16) and pool (7.69 s at T=16) mode. From the pool numbers I can separate the two costs: create+join is roughly 80 µs per thread, one barrier crossing through the pool is roughly 12 µs, and the whole per-image forward pass is only 18 µs – so synchronization costs far more than the work it is supposed to parallelize, for any decomposition, once the unit of work is this small.

Training (MNIST, 10 000 images, 1 epoch, image ownership, create per batch). At batch 32: sequential 0.4292 s, best is $T = 4$ at 0.3074 s ($S_T = 2.01$, 1.40× vs sequential); $T = 16$ regresses to 0.4435 s ($S_T = 1.39$). The serial reduction grows from 0.0068 s (T=1) to 0.0867 s (T=16), and the run creates about 5000 threads total (≈ 313 batches × 16 workers). At batch 256: sequential 0.4272 s, $T = 16$ gives 0.1677 s ($S_T = 3.14$, 2.55× vs sequential), and the reduction only grows to 0.0096 s, because there are 8× fewer batches.

7.3 Results

T	v1 create (s)	v1 pool (s)	v2 create (s)	v2 pool (s)
1	1.1433	0.9101	1.2568	1.0884
4	0.4106	0.4246	0.4609	0.6655
16	0.4869	**0.3205**	0.6011	0.4850

Table 3: Inference, $B = 500$, total median time. Sequential: 0.9153 s.

Two observations I did not verify further, reported only as observations with candidate causes, not as explained facts: (1) in create mode, $T = 1$ is 25–40% slower than the sequential reference, while the pool's $T = 1$ (where the main thread does the work itself) is not – candidate cause: the single created worker thread may land on a slower core or start with cold caches relative to the main thread. (2) v2 with the persistent pool never reaches v2 create's best speedup at $B = 500$ (pool peaks at $S_T = 2.24$ at T=16 vs. create's peak $S_T = 2.73$ at T=4) – candidate cause: v2's dominant costs are the 2 barriers per repeat and the 51 MB batch-wide buffers, not

T	Training batch 32, S_T	Training batch 256, S_T
1	1.00	1.00
4	2.01	2.74
8	1.65	2.91
16	1.39	**3.14**

Table 4: Training speedup vs. $T = 1$ of the same design, two batch sizes.



Figure 1: Inference time vs. T, $B = 500$, all four configurations.

thread creation, so removing thread creation (what the pool does) helps v2 proportionally less than it helps v1.

7.4 Conclusion

Checkpoint – Synthesis 5/5. Main KPIs are the two tables and six figures above (time, speedup, efficiency, throughput and extra memory for every tested T). The trend: both decompositions scale up to $T \approx 4$, then efficiency collapses ($E_T = 0.70$ at T=4 down to 0.18 at T=16 for v1's best inference curve) because the work per worker keeps shrinking while per-batch/per-repeat fixed costs do not. One limiting cost supported by the measurements: the serial gradient reduction, which grows from 0.0068 s to 0.0867 s between $T = 1$ and $T = 16$ at training batch 32.

Image ownership (v1) wins for a large, divisible batch because it needs only one synchronization point and almost no extra memory; row ownership (v2) is the only design that can even attempt to use more workers than there are images, at the cost of 2 barriers and large shared buffers, but in my measurements that cost is never paid back, even at $B = 1 < T$. Granularity decides training performance directly: batch 256 reaches 3.14× at $T = 16$ while batch 32 peaks at 2.01× and then gets *worse* with more threads, because the one cost that grows with T – the serial reduction over T full-network gradient buffers – is amortized over 8× more useful work per batch at batch 256.

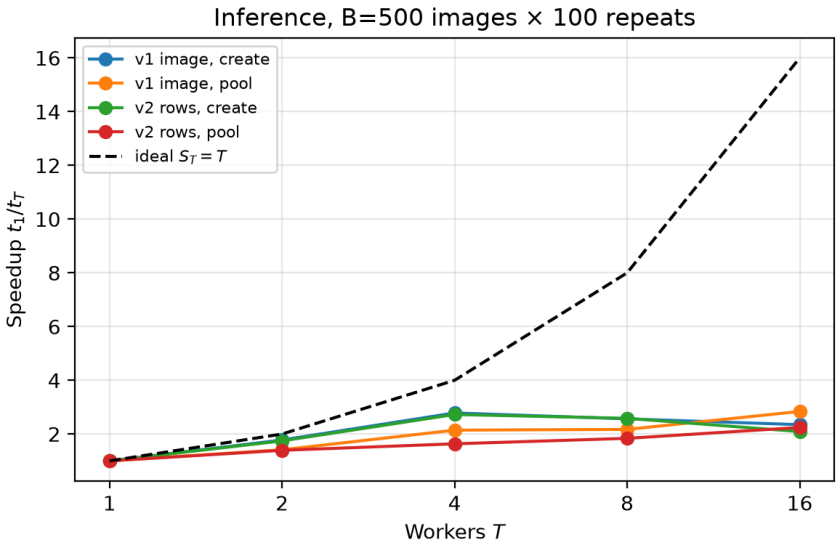


Figure 2: Inference speedup vs. T, $B = 500$.

8 Part F – Interpret the Performance Results

8.1 Objective

Compare the measured training speedups against Amdahl's law with a measured serial fraction φ, and identify which measured costs the model leaves out.

8.2 Answers

Hypothetical exercise. If convolution is 70% of sequential time and becomes exactly 4× faster, the whole-program model speedup is $S_{model} = 1/(0.30 + 0.70/4) \simeq 2.11$, far below 4, because the other 30% of the time does not shrink at all – the whole program is capped at $1/0.30 \approx 3.33\times$ no matter how much faster the convolution gets. With my own measured convolution share of the v1 forward at $T = 1$ ($0.6947/1.1433 = 61\%$), parallelizing only the convolution would cap out around 1.84× at $T = 4$ and 2.33× at $T = 16$; parallelizing the whole forward actually reached 2.84×, which is only possible because more than the convolution is being accelerated.

Measured φ (training, $T = 1$, parallel region create→join divided by the total epoch time). Batch 32: $0.6013/0.6175 = 0.974$. Batch 256: $0.5245/0.5271 = 0.995$. Both are close to 1 because at $T = 1$ almost the whole epoch *is* the "parallel region" (one thread doing all the forward/backward work); the serial reduction and the update are a tiny slice at $T = 1$ and only become significant once T grows.

8.3 Results

T	S_{model}, b32	S_{meas}, b32	S_{model}, b256	S_{meas}, b256
2	1.95	1.59	1.99	1.75
4	3.71	2.01	3.94	2.74
8	6.76	1.65	7.73	2.91
16	11.48	1.39	14.88	3.14

Table 5: Amdahl's model with the measured φ vs. the measured speedup.

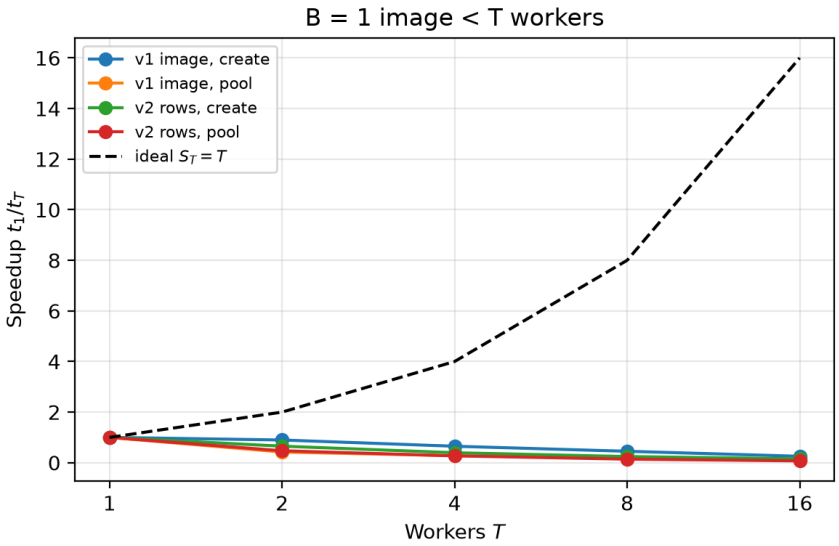


Figure 3: The $B = 1 < T$ case: no configuration speeds up past $T = 1$.

Batch	$T = 2$	$T = 4$	$T = 8$	$T = 16$
32	0.26	0.33	0.55	0.70
256	0.14	0.15	0.25	0.27

Table 6: Effective serial fraction $f = (1/S - 1/T)/(1 - 1/T)$ fitted from each measured S. In the Amdahl model f would be constant.

The model badly over-predicts at every $T > 1$, and the gap grows with T: at batch 32, f climbs from 0.26 to 0.70 as T goes from 2 to 16, instead of staying at φ's complement (≈ 0.026). Measured costs the model leaves out, checked directly against Section 7 and KPI_E.md: (1) the serial reduction grows from 0.0068 s to 0.0867 s between $T = 1$ and $T = 16$ at batch 32 (barely grows at batch 256, 0.0009 s to 0.0096 s); (2) thread creation itself happens *inside* the "parallel region" I timed – about 5000 threads over the epoch at batch 32, $T = 16$ – so the measured φ already includes some serial-looking thread-management cost, and the true parallel fraction is slightly lower than 0.974/0.995; the parallel region alone speeds up $0.6013/0.2809 = 2.14\times$ at $T = 4$ but only $0.6013/0.3396 = 1.77\times$ at $T = 16$, i.e. it stops scaling well on its own, which Amdahl's constant-φ assumption cannot express; (3) at batch 32, $T = 16$ gives only 2 images per worker, and the 6P+8E+2LP-E cores are not equal, so a join waits for whichever worker landed on the slowest core; (4) cache traffic – each worker's reduction step reads a 108 880-byte gradient buffer that another core just finished writing, which is exactly the kind of coherence traffic Lab 2's false-sharing experiment showed is expensive, just at object granularity instead of per-scalar; turbo is also not accounted for by the model, so t_1 may run at a higher clock than t_T. The same code and the same φ reach $3.14\times$ at batch 256 but only $1.39\times$ at batch 32 purely because batch 256 gives $8\times$ more useful work per synchronization point – this is the Part E granularity lesson showing up again, directly in the gap between model and measurement.

8.4 Conclusion

Amdahl's law with a measured φ is the right first model, but it assumes the only cost is the part that stays sequential, and here several of the costs that actually dominate – the reduction, the per-batch thread creation, the core imbalance – all *grow with* T instead of staying fixed. That is why the effective serial fraction I fit back from the measurements keeps climbing with T instead

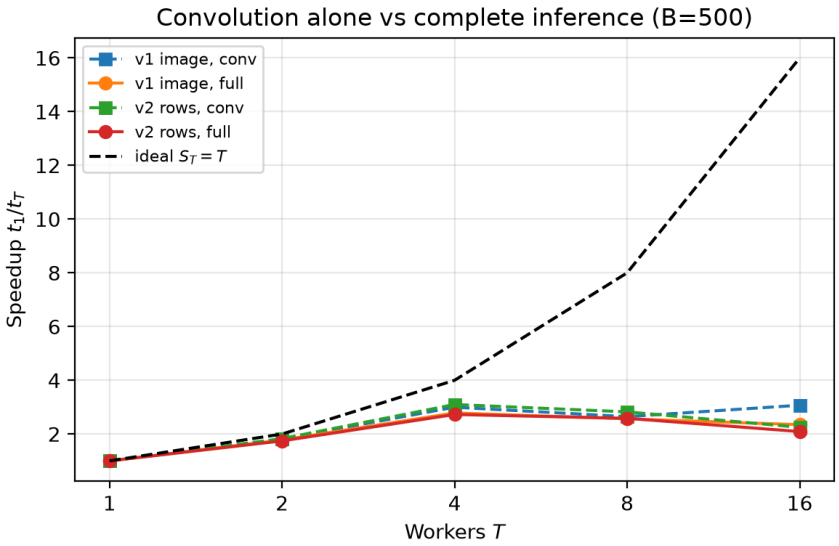


Figure 4: Convolution-only time vs. full-forward time, v1 create.

of staying flat, and why the honest conclusion is not "φ was measured wrong" but "the model is missing terms that this implementation actually pays."

9 Optional Extension – Loop Ordering and Tiling

Not attempted. This was listed as optional in Section 9 of the PDF; I focused the available time on the required Parts B–F instead.

10 Conclusion

What worked: a bit-identical forward pass in both decompositions (image and row ownership), validated parallel training with a worst-case gradient error of 3.8×10^{-15}, and real speedups on the required measurements – 2.86× for inference and 2.55× for training against the sequential reference.

What did not scale: the $B = 1 < T$ case, in every decomposition and every threading mode, because synchronization (tens of microseconds per thread or per barrier crossing) costs far more than an 18-microsecond image; and batch-32 training past $T \approx 4$, because with only 2 images per worker at $T = 16$ the serial reduction over T full-network gradient buffers grows faster than the useful work shrinks.

What I would improve next: use the persistent pool for training too, not just inference (it already removes most of the thread-creation cost in Part E's inference numbers); try a lighter or spinning barrier instead of the mutex-based one for the small-task case; make the reduction itself parallel or tree-shaped instead of one thread walking T buffers serially; and, if I revisited scheduling, try the dynamic (atomic-counter) chunk assignment from Part B's optional extension to deal with the unequal P/E/LP-E cores instead of my fixed equal split.

GitHub Repository

The full source code for this lab is available at: https://github.com/BoraYalcinn/Multithreading-Multicore-Architecture

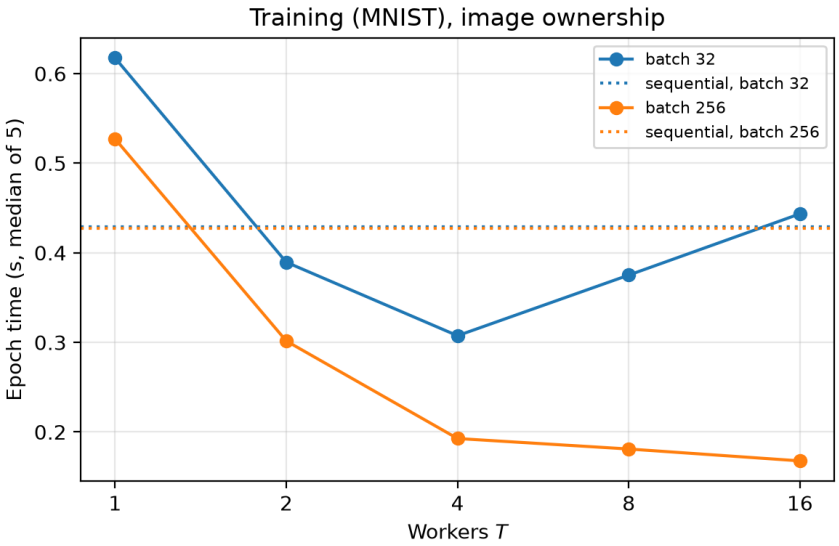


Figure 5: Training epoch time vs. T, batch 32 and batch 256.

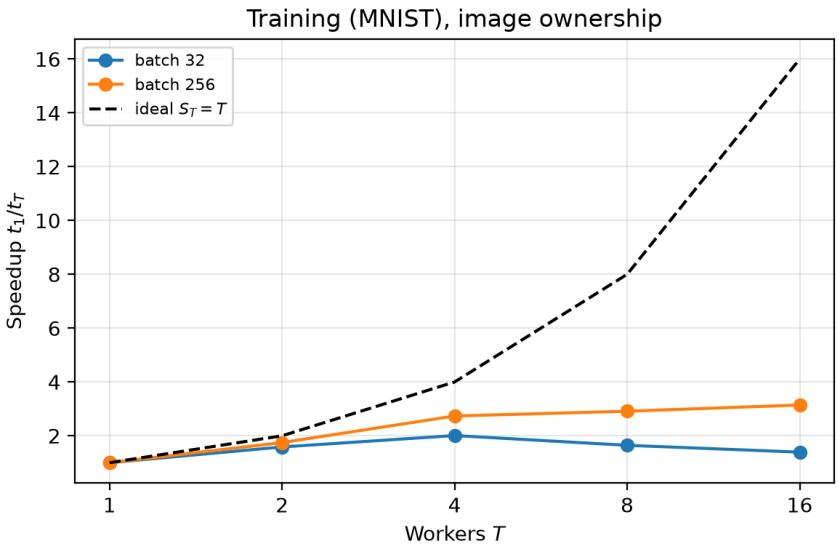


Figure 6: Training speedup vs. T, batch 32 and batch 256.